

Programação Avançada

2024/2025

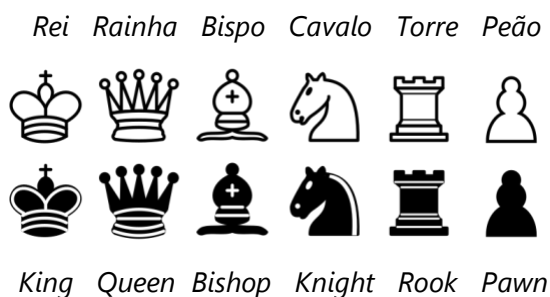
Enunciado do Trabalho Prático

Um dos jogos mais conhecidos em todo o mundo é o *Jogo de Xadrez*. Pretende-se, no contexto do trabalho prático de Programação Avançada desenvolver, de forma progressiva, uma versão do *Jogo de Xadrez* em linguagem *Java*. O desenvolvimento deverá tirar partido do paradigma da Programação Orientada aos Objetos, da tecnologia *JavaFX* e de padrões arquiteturais e funcionais aplicados ao desenvolvimento de software. Este trabalho também é uma oportunidade para quem não conhece o jogo ou não sabe jogar, aprender um dos jogos mais icónicos.

As regras completas do jogo encontram-se disponíveis na *Federação Portuguesa de Xadrez*, em: <https://fpx.pt/site/public/uploads/formacao/Leis-do-Xadrez-Atualizadas-Viana-Brandao.pdf>

Resumidamente, o jogo é disputado por dois jogadores, um com 16 peças brancas e outro com 16 peças pretas, e desenrola-se num tabuleiro com 8 x 8 (64 casas). Cada jogador possui 1 x *Rei*, 1 x *Rainha*, 2 x *Bispo*, 2 x *Cavalo*, 2 x *Torre*, 8 x *Peão*.

A aparência visual das peças e a sua disposição inicial no tabuleiro é a seguinte:



Os jogadores vão alternando a vez de jogar, cabendo ao jogador com as peças brancas efetuar a primeira jogada. Os movimentos possíveis são:

- *Rei* – Uma casa em qualquer direção (horizontal, vertical e diagonal);
- *Rainha* – Qualquer número de casas em qualquer direção;
- *Bispo* – Qualquer número de casas na diagonal;
- *Cavalo* – Movimento em "L": 2 casas na horizontal ou vertical seguidas de um movimento de uma casa na perpendicular ao sentido do movimento inicial;

- *Torre* – Qualquer número de casas na vertical ou horizontal;
- *Peão* – Uma casa no sentido vertical na direção da posição inicial das peças do adversário. Na primeira jogada de cada peão, este pode mover-se duas casas de uma vez só.

Adicionalmente, há que ter em consideração que não é permitido jogar para uma casa onde se encontra uma peça do mesmo jogador que efetua a jogada e os movimentos não podem ultrapassar os limites do tabuleiro. Em cada movimento de múltiplas casas, há também que garantir que todas as casas no percurso seguido se encontram livres, exceto para o movimento do *Cavalo*, em que só interessa a última casa. Se na casa destino estiver uma peça do adversário, então essa peça é capturada. No caso dos *Peões*, a captura das peças do adversário é realizada caso elas se encontrem à distância de uma casa na diagonal, no sentido do movimento do peão.

O objetivo do jogo é capturar a peça *Rei* do adversário, nesse momento o jogo acaba. Não é permitido movimentar uma peça que deixe o próprio *Rei* em risco de ser capturado.

Poderão consultar as regras completas no documento referido, onde podem encontrar mais informação sobre os movimentos permitidos, incluindo movimentos especiais como o "*Roque*", "*Captura em passagem*" ("*en passant*") e "*Promoção de um peão*". Ter também em atenção as situações que podem conduzir ao empate do jogo.

Desenvolvimento

O trabalho será desenvolvido em grupos, com no máximo 3 elementos (de acordo com a constituição de grupo comunicada até final de Fevereiro). As diversas etapas de realização do trabalho serão comunicadas e iniciadas nas aulas práticas. Os grupos constituídos por alunos com o estatuto de trabalhador-estudante ou similar deverão acompanhar com atenção as informações semanais que vão ser disponibilizadas (por exemplo: slides das aulas) para realização das tarefas que forem sendo definidas.

Como referido, as tarefas serão preferencialmente iniciadas em sala de aula e deverão ser finalizadas até ao dia anterior à aula em que são avaliadas. No caso dos trabalhadores-estudantes que optaram pela modalidade de avaliação alternativa, a verificação e avaliação das etapas semanais será efetuada tendo por base um relatório a ser entregue até ao final do dia de Sábado da semana seguinte ao lançamento da tarefa. Os alunos trabalhadores-estudantes poderão ser convocados para esclarecimento de dúvidas sobre a realização da tarefa, sendo para isso marcada uma reunião (presencial ou remota) em data a combinar com o grupo.

Para auxiliar as tarefas de gestão do desenvolvimento em grupo, registo da evolução e acompanhamento pelos docentes, será usado o *GitHub+GitHub Classroom*. Na primeira aula prática será fornecido um *link* que permitirá aos elementos dos grupos registarem-se e serem associados a um grupo.

A cada etapa de evolução do trabalho, o grupo deverá criar um novo *branch* para realização das tarefas. O *branch* deverá ter a designação que será comunicada semanalmente. Até à data de entrega da tarefa deverá ser feito um *Pull Request* e um *Merge* (preferencialmente *Squash and Merge*) para o *branch* main.

Até ao final do dia 2 de Junho todos os grupos (independentemente do tipo de grupo) deverão entregar um relatório final do trabalho. Os grupos com avaliação "*Normal*" deverão preparar uma apresentação da versão final do trabalho e respetivo relatório. A apresentação não poderá exceder

7.5 minutos, seguindo-se a defesa das opções tomadas. Estas defesas serão realizadas nos horários normais das aulas práticas.

Para realização das defesas dos trabalhos desenvolvidos sem avaliação em aula (trabalhadores-estudantes ou similares) serão marcadas sessões presenciais a realizar durante a última semana de aulas. Estas sessões incluirão a apresentação, demonstração das funcionalidades e defesa, tendo uma duração de +/-30 minutos, sendo obrigatório que todos os elementos do grupo estejam presentes.

Como já referido, o desenvolvimento do trabalho prático será realizado em diversas etapas, cada uma com início numa aula prática, prevendo-se a seguinte calendarização:

- [2025.03.26-28] Definição do modelo de dados – Definição da classe tabuleiro, da classe peça base e das suas derivadas
- [2025.03.02-04] Gestão de jogo e persistência dos dados – Classes para representar um jogo e os jogadores, gestão da persistência dos dados através de serialização, importação e exportação de jogos parciais/incompletos
- [2025.04.09-11] Interface gráfica Java FX para gestão da aplicação – Criação da interface gráfica e disponibilização das opções de gestão geral da aplicação
- [2025.04.23-05.02] Interface gráfica Java FX para o jogo – Desenho do tabuleiro de jogo, apresentação das peças através de imagens, seleção de jogadas
- [2025.05.07-09] Incorporação de Facade observável – Organização e funcionamento segundo o padrão arquitetural MVVM
- [2025.05.14-16] Incorporação de modo de aprendizagem de xadrez e acessibilidade – Incorporação de funcionalidade de *undo/redo* e áudio

(Nota: a calendarização apresentada é apenas indicativa, podendo vir a ser ajustada às necessidades de leção de conteúdos que suportem as funcionalidades previstas)

Notas sobre o trabalho final a submeter

A forma de implementação dos padrões é muito diversa, pelo que se deve restringir às formas de implementação usadas no contexto das aulas. Outras formas de implementação não serão consideradas nem avaliadas.

O código fonte da aplicação deve apresentar uma divisão clara entre o modelo e a *UI*. O código do modelo não deve conter qualquer tipo de interação com o utilizador e a interface com o utilizador não deve incluir qualquer tipo de lógica da simulação. O código da interface com o utilizador poderá incluir lógica de pré-tratamento das ações do utilizador (por exemplo, filtrar ou pré-processar os *inputs* do utilizador), mas o modelo deve ser robusto de modo a atuar nas ordens e receção de dados incorretos ou fora de contexto. A estrutura do projeto, em termos de *packages*, deve refletir esta separação, conforme explicado mais à frente.

A atualização da interface com o utilizador deverá estar de acordo com o padrão de notificações assíncronas estudado nas aulas. Espera-se que as interfaces com o utilizador sejam apelativas, intuitivas e funcionais.

As classes de todo o modelo de dados, incluindo a *Facade*, deverão possuir comentários compatíveis com a geração de documentação usando *JavaDoc*. A documentação gerada não

necessita ser entregue com o trabalho, mas deverão estar aptos a gerá-la durante a defesa do trabalho.

As classes do modelo de dados que incluam algoritmos e a classe que implementa o padrão *Facade*, deverão incluir testes unitários (*JUnit*). Cada uma dessas classes deverá ter 2 testes (mínimo).

No desenvolvimento desta aplicação não poderá recorrer a bibliotecas externas não referidas explicitamente no contexto deste enunciado ou das aulas.

Estrutura de projeto

A aplicação deve estar organizada em *packages*, incluindo pelo menos os seguintes (podem existir outros):

- **pt.isec.pa.chess** – *package* que abrange toda a aplicação;
- **pt.isec.pa.chess.model** – inclui a classe que constitui a *Facade* de acesso à lógica, que internamente distribui as responsabilidades que lhe são pedidas;
- **pt.isec.pa.chess.model.data** – contém as classes que representam as estruturas de dados e respetiva gestão;
- **pt.isec.pa.chess.ui** – classes que implementam a interface em modo gráfico em *Java Fx*;
- **pt.isec.pa.chess.ui.res** – *package* base para gestão dos recursos multimédia da aplicação, recorrendo, por exemplo, ao padrão *multiton*.

Na raiz do diretório de projeto, para além da pasta *src* com os *packages* referidos, deverão existir os seguintes diretórios:

- **reports** – relatórios produzidos: relatórios por tarefa (trabalhadores-estudantes ou similares) e relatório final (todos os alunos);
- **test** – todas as classes para realização dos testes unitários, respeitando a hierarquia de *packages* referida anteriormente.

O relatório final deve incluir:

- Uma descrição sintética acerca das opções e decisões tomadas na implementação;
- Diagramas de padrões de programação aplicados no trabalho;
- Tabela com descrição sucinta das classes utilizadas no programa (o que representam e os objetivos);
- Descrição sucinta do relacionamento entre as classes (podem ser usados diagramas UML);
- Para cada funcionalidade esperada da aplicação, a indicação de cumprido/implementado totalmente ou parcialmente (especificar o que foi efetivamente cumprido neste caso) ou não cumprido/implementado (especificar a razão). O uso de uma tabela pode simplificar a elaboração desta parte.